

Midterm Exam

Ahmad Mousavi

2024-02-18

Respond in an R Markdown file and have each question in one section!

1. Create a function (call it `sum_positive_comp`) that accepts two vectors as inputs and omits all the not available (NA) components in each vector. Then, looks at the NA-omitted vectors and if all the components of either of these vectors are negative, the functions stops and gives the message “one of the inputs doesn’t have any positive elements”. Otherwise, it compares the summation of positive components of the first vector with the summation of positive components of the second vector. It return TRUE if the summation of the positive components of the first one is bigger than equal to the summation of positive components of the second one. Otherwise it returns FALSE.

Make sure it works for the following set of inputs:

- `inp_1 = c(-1,10,NA)` and `inp_2 = c(-1,0,4,5,NA)` returns TRUE
- `inp_1 = c(-10:-1)` and `inp_2 = c(-1)` returns “one doesn’t have any positive components”

```
# Here is the code:

sum_positive_comp <- function(inp_1, inp_2) {

  # Omit the NA components
  inp1 <- na.omit(inp_1)
  inp2 <- na.omit(inp_2)

  # Check if both of them have at least one positive component
  if (all(inp1 <= 0) || all(inp2 <= 0)) {
    return("one doesn't have any positive components")
  }

  # Calculate the summation of positive components of each vector
  sum_pos_inp1 <- sum(inp1[inp1 > 0])
  sum_pos_inp2 <- sum(inp2[inp2 > 0])

  # Compare the summation of positive components
  if (sum_pos_inp1 > sum_pos_inp2) {
    return(TRUE) # The condition is met
  } else {
    return(FALSE) # The condition is not met
  }
}
```

```
inp_1 <- c(-1,10,NA)
inp_2 <- c(-1,0,4,5,NA)
result <- sum_positive_comp(inp_1, inp_2)
print(result)
```

```
## [1] TRUE
```

```
inp_1 = c(-10:-1)
inp_2 = c(-1)
result <- sum_positive_comp(inp_1, inp_2)
print(result)
```

```
## [1] "one doesn't have any positive components"
```

2. Create the following (call it df):

```
df <- data.frame(names = c("Tina", "Mahin", "Sol", "Pershia", "Dorsa"),
                 age = c(18, 29,20,30,24), income = c(NA, 4000, 3000, 2500,NA),
                 happiness = c(NA, 9, 7, Inf,9))
df
```

```
##      names age income happiness
## 1    Tina  18     NA          NA
## 2   Mahin  29    4000           9
## 3     Sol  20    3000           7
## 4 Pershia  30    2500          Inf
## 5   Dorsa  24     NA           9
```

- Create the following subset (call df_1):

```
df_1 <- df[df$age > 25,1:3]
df_1
```

```
##      names age income
## 2   Mahin  29    4000
## 4 Pershia  30    2500
```

- compute the mean of income for those whose age is more than 25 (directly from df)?

```
mean_1 <- mean(df$income[df$age>25])
mean_1
```

```
## [1] 3250
```

- compute the mean of income for those whose age is more than 25 (directly from df_1)?

```
mean_2 <- mean(df_1$income)
mean_2
```

```
## [1] 3250
```

- find the sum of finite values in the happiness column?
- who are the happiest people (more than average)?

```
vec_log <- !is.na(df$happiness) & is.finite(df$happiness)
mean_3 <- mean(df$happiness[vec_log])
df$name[df$happiness > mean_3 & !is.na(df$happiness)]
```

```
## [1] "Mahin" "Pershia" "Dorsa"
```

3. Using only dplyr functions:

- Report the number of rows in which both “dep_delay” and “distance” are NA!

```
flights %>%
  filter(!is.na(dep_delay), !is.na(distance)) %>%
  summarise(n = n())
```

```
## # A tibble: 1 x 1
##       n
##   <int>
## 1 328521
```

- Report the number of rows in which there exists at least one NA.

```
flights %>%
  filter(rowSums(is.na(.)) >= 1) %>%
  summarise(n = n())
```

```
## # A tibble: 1 x 1
##       n
##   <int>
## 1  9430
```

- filter rows with only NA values.

```
flights %>%
  filter(rowSums(is.na(.)) == ncol(flights))
```

```
## # A tibble: 0 x 19
## # i 19 variables: year <int>, month <int>, day <int>, dep_time <int>,
## #   sched_dep_time <int>, dep_delay <dbl>, arr_time <int>,
## #   sched_arr_time <int>, arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- Filter all the rows with more than and equal to 2 NA's and less than equal to 5 NA's:

```
flights %>%
  filter(rowSums(is.na(.)) >= 2, rowSums(is.na(.)) <= 5)
```

```
## # A tibble: 6,918 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>         <dbl>   <int>         <int>
## 1  2013     1     1    1525           1530         -5    1934           1805
## 2  2013     1     1    1528           1459         29    2002           1647
## 3  2013     1     1    1740           1745         -5    2158           2020
## 4  2013     1     1    1807           1738         29    2251           2103
## 5  2013     1     1    1939           1840         59      29           2151
## 6  2013     1     1    1952           1930         22   2358           2207
## 7  2013     1     1    2016           1930         46      NA           2220
## 8  2013     1     1      NA           1630         NA      NA           1815
## 9  2013     1     1      NA           1935         NA      NA           2240
## 10 2013     1     1      NA           1500         NA      NA           1825
## # i 6,908 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

- Report the row numbers for each row and the number of NA's in them (Only the first ten rows).

```
flights %>%
  summarise(Row_Number = row_number(), NAsCounts = rowSums(is.na(.))) %>%
  ungroup() %>%
  slice_max(NAsCounts) %>%
  slice(1:10)
```

```
## Warning: Returning more (or less) than 1 row per 'summarise()' group was deprecated in
## dplyr 1.1.0.
## i Please use 'reframe()' instead.
## i When switching from 'summarise()' to 'reframe()', remember that 'reframe()'
##   always returns an ungrouped data frame and adjust accordingly.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
## # A tibble: 10 x 2
##   Row_Number NAsCounts
##   <int>         <dbl>
## 1     1783          6
## 2     1785          6
## 3     2698          6
## 4     2699          6
## 5     3609          6
## 6     3610          6
## 7     4333          6
## 8     6099          6
## 9     6998          6
## 10    7896          6
```

```
flights %>%
  mutate(Row_Number = row_number()) %>%
  mutate(NAsCounts = rowSums(is.na(.))) %>%
  transmute(Row_Number, NAsCounts) %>%
  arrange(-NAsCounts) %>%
  slice(1:10)
```

```
## # A tibble: 10 x 2
##   Row_Number NAsCounts
##   <int>      <dbl>
## 1      1783         6
## 2      1785         6
## 3      2698         6
## 4      2699         6
## 5      3609         6
## 6      3610         6
## 7      4333         6
## 8      6099         6
## 9      6998         6
## 10     7896         6
```

- Report the number of NA's in each column!

```
flights %>%
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
## # A tibble: 1 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <int> <int> <int>   <int>         <int>      <int>   <int>         <int>
## 1     0     0     0    8255             0      8255    8713             0
## # i 11 more variables: arr_delay <int>, carrier <int>, flight <int>,
## #   tailnum <int>, origin <int>, dest <int>, air_time <int>, distance <int>,
## #   hour <int>, minute <int>, time_hour <int>
```

- Report the number of distinct rows.

```
flights %>%
  summarize(distinct_rows = nrow(distinct(.)))
```

```
## # A tibble: 1 x 1
##   distinct_rows
##   <int>
## 1      336776
```

5. Set seed to be 2023. Create a column called “price” with numbers between 100 to 1300, and add it to the end of “flights” dataset. Use normal distribution with mean 700 and standard deviation 600! Add a noise based on normal with mean 0 and standard deviation 1.

```
set.seed(2023)
p <- rnorm(nrow(flights), mean = 700, sd = 600)
p <- pmax(100, pmin(1300, p))
p <- p + rnorm(nrow(flights), mean = 0, sd = 1)
flights$price <- p
```

6. Find the cheapest and the most expensive airlines, based on the average ticket prices, for traveling from any airport in New York to Washington DC (IAD and DCA) during the third week of December.

```
flights %>%
  filter(origin %in% c("EWR", "JFK", "LGA"),
         dest %in% c("IAD", "DCA"),
         month == 12,
         day %in% 15:21) %>%
  slice_min(price, n=1) -> Q1_min
print(paste(Q1_min$carrier, Q1_min$price))
```

```
## [1] "US 97.9175146611989"
```

```
flights %>%
  filter(origin %in% c("EWR", "JFK", "LGA"),
         dest %in% c("IAD", "DCA"),
         month == 12,
         day %in% 15:21) %>%
  slice_max(price, n=1) -> Q1_max
print(paste(Q1_max$carrier, Q1_max$price))
```

```
## [1] "EV 1302.18028823297"
```

7. Discover the priciest flight from EWR to LAX that departs with no more than a 15-minute delay, specifically during the latter half of August.

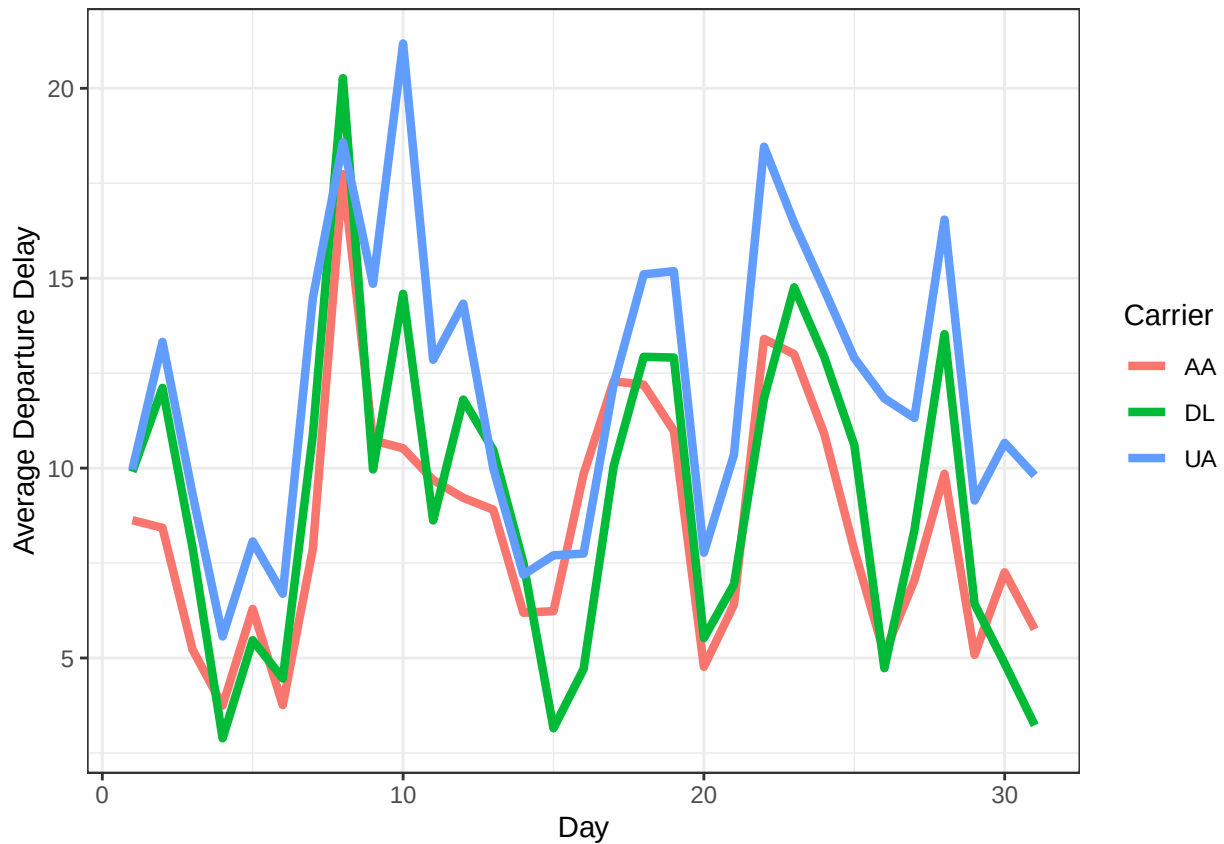
```
flights %>%
  filter(origin == "EWR",
         dest == "LAX",
         dep_delay <= 15,
         month == 8,
         day <= 15) %>%
  arrange(-price) %>%
  slice(1) -> Q2
print(Q2$carrier)
```

```
## [1] "UA"
```

8. For airlines “AA”, “DL”, and “UA”, plot the mean of departure delay against each day!

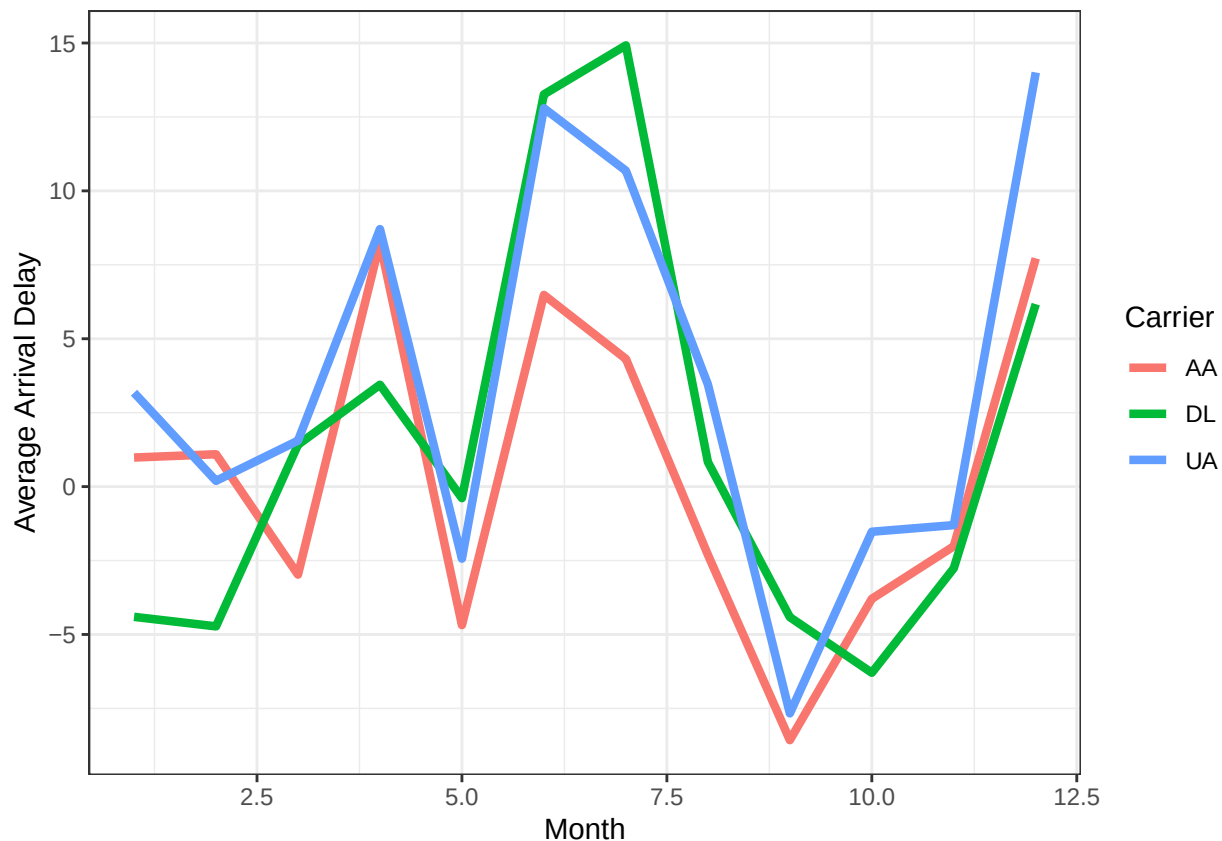
```
flights %>%
  filter(!is.na(carrier), !is.na(day), !is.na(dep_delay)) %>%
  filter(carrier %in% c("AA", "DL", "UA")) %>%
  group_by(carrier, day) %>%
  summarise(across(.cols = dep_delay, .fns = list(~ mean(.x))), .groups = "drop") %>%
  ggplot(aes(day, dep_delay_1, color = carrier)) +
  geom_line(size = 1.5) +
  xlab("Day") +
  ylab("Average Departure Delay") +
  scale_color_discrete("Carrier") +
  theme_bw()
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



9. For airlines “AA”, “DL”, and “UA”, plot the average of arrival delay against each month. Which two months seem to have the least arrival delays?

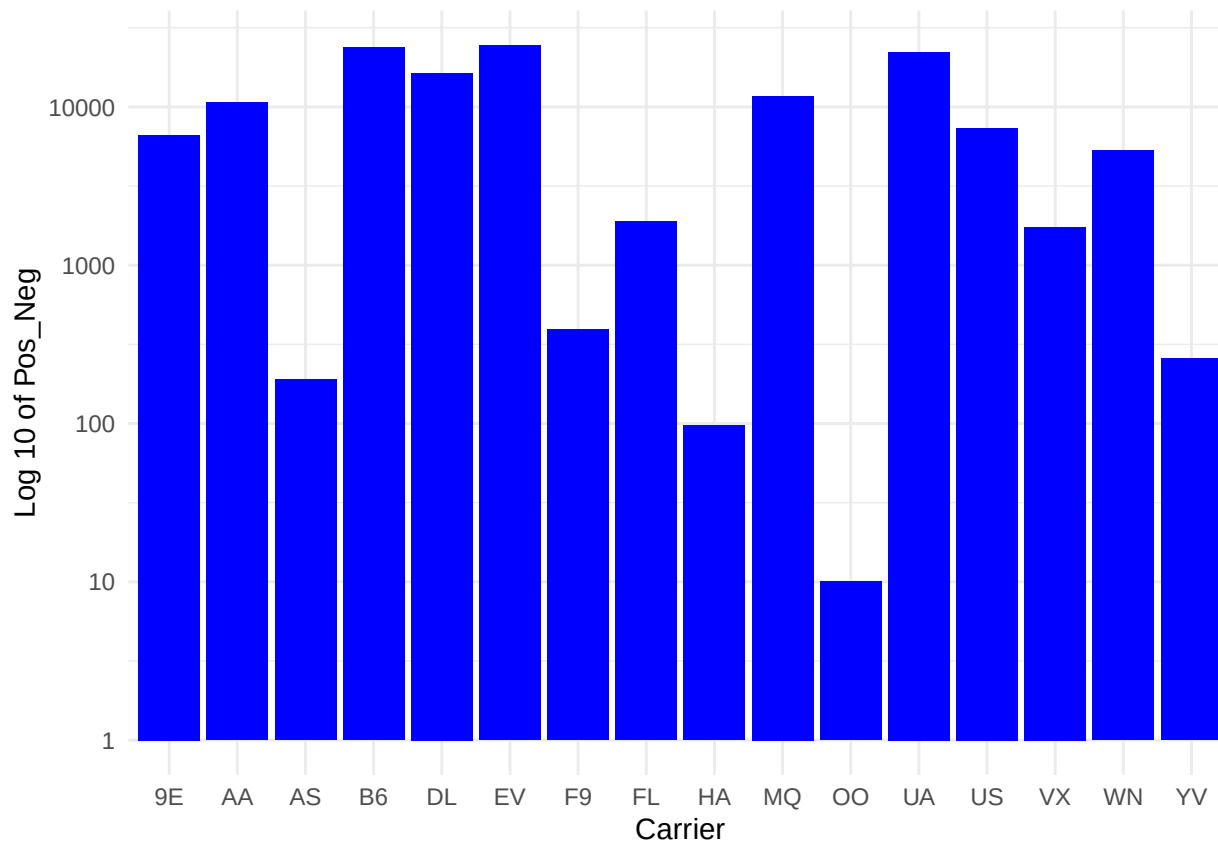
```
flights %>%
  filter(!is.na(carrier), !is.na(month), !is.na(arr_delay)) %>%
  filter(carrier %in% c("AA", "DL", "UA")) %>%
  group_by(carrier, month) %>%
  summarise(across(.cols = arr_delay, .fns = list(~ mean(.x))), .groups = "drop") %>%
  ggplot(aes(month, arr_delay_1, color = carrier)) +
  geom_line(size = 1.5) +
  xlab("Month") +
  ylab("Average Arrival Delay") +
  scale_color_discrete("Carrier") +
  theme_bw()
```



10. Write a function, which takes a vector and removes all the NA values and then outputs the difference of the number of positive values minus the number of negative values in that vector. Call it `pos_neg`! Make sure that your function works on `c(2,3,NA,NA,10,-4,NA)`. Next, for each airline, plot the (log 10 of) `pos_neg` of arrival delay column (use `geom_bar`).

```
pos_neg <- function(x){
  x <- x[!is.na(x)] # drop NA values
  return(length(x[x>0]) - length(x[x<0]))
}

flights %>%
  filter(!is.na(carrier), !is.na(arr_delay)) %>%
  group_by(carrier) %>%
  summarise(pos_neg_arr = pos_neg(arr_delay), .groups = "drop") %>%
  ggplot(aes(carrier, pos_neg_arr)) +
  scale_y_log10() +
  geom_bar(stat = "identity", fill = "blue") +
  labs(x = "Carrier", y = "Log 10 of Pos_Neg") +
  theme_minimal()
```



11. First, remove all entries with missing information in the 'delay' and 'carrier' columns. Next, focus on flights with a duration between 2 and 5 hours. Determine which airline offers the most economical fares for this category of flights. Additionally, explore if the same airline remains the cheapest when considering flights with duration of less than 2 hours.
12. Identify the airline with the highest average departure delay for flights originating from San Francisco International Airport (SFO) during the summer months (June, July, August). Additionally, find the average departure delay for this airline during this period.
13. Create a column that is the average of all the numerical columns in flights data frame. Create another column that is the average of the numerical columns.

```
# efficient
flights %>%
  mutate(sum_of_numeric_cols = rowSums(across(where(is.numeric)))) %>%
  mutate(mean_of_numeric_cols = rowMeans(across(where(is.numeric)))) %>%
  relocate(sum_of_numeric_cols, mean_of_numeric_cols)
```

```
## # A tibble: 336,776 x 22
##   sum_of_numeric_cols mean_of_numeric_cols year month   day dep_time
##           <dbl>           <dbl> <int> <int> <int>   <int>
## 1             8550.             1069.  2013     1     1       517
## 2             8280.             1035.  2013     1     1       533
## 3             7439.              930.  2013     1     1       542
## 4             8234.             1029.  2013     1     1       544
## 5             6452.              806.  2013     1     1       554
```

```
## 6      8532.      1066. 2013    1    1    554
## 7      6838.      855. 2013    1    1    555
## 8     11885.     1486. 2013    1    1    557
## 9      6476.      809. 2013    1    1    557
## 10     6275.      784. 2013    1    1    558
## # i 336,766 more rows
## # i 16 more variables: sched_dep_time <int>, dep_delay <dbl>, arr_time <int>,
## #   sched_arr_time <int>, arr_delay <dbl>, carrier <chr>, flight <int>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>, price <dbl>
```

```
# inefficient
# flights %>%
#   rowwise() %>%
#   mutate(cum_columns = mean(c_across(where(is.numeric))))
```